Phase 2 Learning Guide: Building the Bronze and Silver Layers

This guide explains what we built in Phase 2, why each piece is designed the way it is, and how to run and demonstrate it. Read it top to bottom once. After that, use the requirement map in section 2 to jump to whatever you need to explain.

Project: Stack Overflow Developer Q&A Lakehouse · **Team:** Muhammad Sufyan (24L-2601), Muaaz Fahad (24L-2563)

---

1. The big picture

Phase 1 produced a plan and raw sample files. Phase 2 turns that into a real pipeline that a company could run every day without anyone watching it. The instructor's requirements boil down to five ideas:

Idea	Plain meaning
Contracts	We decide the exact shape of the data up front. Spark is never allowed to guess.
Idempotency	Running the same job twice on the same data gives the same result as running it once.
Parameterization	Any day, batch or folder can be re-processed on demand. Nothing is hardcoded to "today".
Fault tolerance	One bad record or one bad file does not crash the whole run.
Auditability	Every run leaves a record of what it did, when, and how many rows it touched.

The flow

```
Stack Exchange API
   |   ingestion/full_load.py, ingestion/incremental_load.py
   v
Landing volume   /Volumes/workspace/so_raw/landing/{full|incremental}/<batch_id>/
   |   notebooks/01_raw_to_bronze  ->  so_lakehouse/bronze.py
   v
Bronze   so_bronze.*   raw records typed by the contract, one row per record per batch
   |   notebooks/02_bronze_to_silver  ->  so_lakehouse/silver.py
   v
Silver   so_silver.*   one clean, current, PII-safe row per entity, kept with MERGE INTO
```

Next to the data, three **operational tables** in `so_ops` record what happened: `pipeline_execution_logs`, `batch_registry` and `schema_drift_events`.

Why a Python package plus thin notebooks?

All the logic lives in `src/so_lakehouse/`. The notebooks only read widget values and call one function. That gives three benefits:

1. **One implementation, many entry points.** The notebooks, the Databricks job, the command line and the tests all call the same code.
2. **Testable.** `tests/test_pipeline.py` runs the real functions on local Spark, so every requirement is proven automatically.
3. **Readable in Git.** Plain `.py` files show clean diffs in pull requests. Notebooks are saved in Databricks "source" format for the same reason.

---

2. Requirement map

#	Instructor requirement	What we did	Code
1	Workspace	Databricks Free Edition with Unity Catalog; tables in catalog `workspace`	`notebooks/00_setup.py`, `ddl.py`
1	Continuous Git commits	Everything is in the GitHub repo; the workspace uses a Git folder	repo history
2	Data dictionary	Generated from the schema code into README and `docs/data_dictionary.md`	`scripts/generate_data_dictionary.py`
2	Strict schema-on-read	Every file is parsed with an explicit `StructType`; no `inferSchema` anywhere, enforced by a test	`schemas.py`, `bronze.py`

#	Instructor requirement	What we did	Code
2	Casting	Epoch integers become `TIMESTAMP`; every Silver column is cast to its contract type	`silver.py: ts(), conform()`
2	`load_timestamp` everywhere	A column in all 15 tables, set when each row is written	`schemas.py`
3	Idempotent, `MERGE INTO`	Every Silver table is upserted with `MERGE INTO`; Bronze replaces a batch's own rows	`silver.py`, `bronze.py`
3	Parameterized backfills	`mode`, `batch_ids`, `start_date`, `end_date`, `landing_path`	`batches.py`, notebooks 01 and 02
3	Schema drift	New columns evolve the table (`mergeSchema`); type changes and bad lines go to quarantine	`drift.py`, `bronze.py`
4	Logging tables	`so_ops.pipeline_execution_logs`, plus a batch registry and drift events	`audit.py`
4	Audit metrics	Layer, parameter, start, end, status, rows read, inserted, updated, deleted, quarantined	`audit.py`

3. The environment

Databricks Free Edition

Databricks Free Edition replaced the old Community Edition. Three of its features matter for us:

- **Unity Catalog** names everything with three levels: `catalog.schema.table`. Our tables are `workspace.so_bronze.questions`, `workspace.so_silver.questions` and so on.
- **Volumes** are governed folders for files. Raw batches live in the volume `workspace.so_raw.landing`, which notebooks can read like a normal folder at `/Volumes/workspace/so_raw/landing`.
- **Serverless compute** runs notebooks without us managing a cluster. It is built on **Spark Connect**, which has no `SparkContext` and no RDD API. Our code therefore uses only DataFrame and SQL APIs. For example, it reads the Delta log with `DESCRIBE HISTORY` instead of internal Java objects.

Local development

The same package runs on a laptop with open-source Spark 3.5 and Delta Lake 3.2 (`src/so_lakehouse/spark_session.py`). That is how the tests run. Locally there is no catalog, so tables have two-level names such as `so_silver.questions`. `PipelineConfig` hides that difference:

```
cfg = PipelineConfig.for_databricks(catalog="workspace")   # workspace.so_silver.questions
cfg = PipelineConfig.for_local("data/demo_landing")         # so_silver.questions
cfg.table("silver.questions")
```

4. Landing zone and batches

The ingestion scripts write one folder per extraction, called a **batch**:

```
landing/
  full/
    full_2025-01-01_2025-04-01/      questions.jsonl  answers.jsonl  _manifest.json
    full_2025-04-01_2025-07-01/
  incremental/
    incr_20260925T155859Z/           questions.jsonl  answers.jsonl  deleted_questions.jsonl
    incr_20261007T155914Z/           question_snapshots.jsonl  _manifest.json
    incr_20261007T160033Z/
```

- **Batch id** = the folder name. It is deterministic, so reloading the same folder always maps to the same id, which is what makes Bronze idempotent.
- **Manifest** = `_manifest.json`, written by the extractor. It records the extraction time, the window and the record counts. The pipeline reads `extracted_at_utc` from it: that becomes `_extracted_at` on every Bronze row, and its date becomes the **batch date** used for date-range backfills.
- **Immutable files.** A batch folder is never edited after it lands. If a batch must be redone, the extractor writes a new folder.

The demo uses five real batches stored in `data/demo_landing`: two full loads covering January to June 2025, and three incremental pulls from 25 September and 7 October 2026. The two 7 October pulls overlap on purpose, so the

same questions arrive twice. That is the situation idempotency has to survive.

5. Strict schema-on-read

Why not `inferSchema`?

Inference reads a sample of the data and guesses types. That is dangerous in production:

- A column that is empty in the sample becomes `string`, and later batches with numbers silently change meaning.
- Two batches can infer different schemas, and the table breaks.
- Inference scans the data an extra time, which costs compute.

A **contract** fixes the types once, in code, and every batch is checked against it.

Our contracts

`src/so_lakehouse/schemas.py` defines one `StructType` per raw entity. An excerpt:

```
QUESTION_RAW = StructType([
    col("question_id", LongType(), "Question id (natural key)"),
    col("title", StringType(), "Title, HTML-entity encoded"),
    col("tags", ArrayType(StringType()), "All tags on the question"),
    col("owner", OWNER, "Author snapshot"),                    # nested StructType
    col("score", IntegerType(), "Upvotes minus downvotes at extraction time"),
    col("creation_date", LongType(), "Creation time, epoch seconds"),
    ...
    col("posted_by_collectives", StringType(), "Raw JSON array of collectives"),
])
```

`col()` is a tiny helper that stores a description in the field's metadata. The same descriptions become Delta column comments and the data dictionary.

A useful trick: a few deeply nested, rarely used objects, such as the collectives a post belongs to, are declared as `StringType`. When Spark meets a JSON object or array in a `STRING` field, it keeps the raw JSON text. We keep the information without modelling forty sub-fields.

How the file is read

```
parse_schema = StructType(contract.fields + [StructField("_corrupt_record", StringType())])
parsed = (spark.read.text(path)                                  # 1 line = 1 record, no guessing
          .select("value", F.col("_metadata.file_path").alias("_source_file"))
          .withColumn("rec", F.from_json("value", parse_schema,
                      {"mode": "PERMISSIVE", "columnNameOfCorruptRecord": "_corrupt_record"})))
```

- `spark.read.text` gives each raw line as a string. Keeping the original line lets us quarantine it untouched and inspect it for drift.
- `from_json(value, schema)` applies the contract. This is schema-on-read with an explicit schema.
- **PERMISSIVE mode** never throws. If a line cannot be converted, the failing fields become null and the whole line is copied into `_corrupt_record`. The other modes are DROPMALFORMED, which silently loses data, and FAILFAST, which crashes the batch. Neither is acceptable for us.
- `_metadata.file_path` is Spark's built-in file metadata column. We use it instead of `input_file_name()`, which Unity Catalog does not support.

6. The Bronze layer

What a Bronze row contains

Every contract column, plus metadata:

Column	Why
`_batch_id`, `_load_type`	Which batch and load type the row came from
`_batch_date`, `_extracted_at`	When it was extracted; drives backfills and Silver versioning
`_source_file`	Lineage back to the exact file
`_record_hash`	SHA-256 of the raw line
`_rescued_data`	Fields that were not in the contract, as JSON
`load_timestamp`	When this row was written to Bronze

Bronze keeps **history**: the same question appears once per batch that contained it. Silver later picks the newest version.

Idempotent Bronze writes with `replaceWhere`

```
(good.write.format("delta").mode("overwrite")
     .option("replaceWhere", "_batch_id = 'incr_20261007T155914Z'")
     .saveAsTable("workspace.so_bronze.questions"))
```

`replaceWhere` is an overwrite of only the rows matching the predicate. Loading a batch deletes that batch's previous rows and writes the new ones in one atomic Delta transaction. Loading the same batch ten times leaves exactly one copy. Other batches are untouched. The log records how many rows were replaced in `rows_deleted`.

The batch registry

`so_ops.batch_registry` has one row per batch with its Bronze status and time, and its Silver status and time. It is maintained with `MERGE INTO`. It answers "what is new?":

- **Raw-to-Bronze, incremental mode:** load batches whose `bronze_status` is not `SUCCESS`.
- **Bronze-to-Silver, incremental mode:** process batches whose Silver run is missing, failed, or older than the latest Bronze load.

---

7. Schema drift

Sources change without warning. The Stack Exchange API might add a field tomorrow, or a field could change type. Spark's schema-on-read **hides** both: unknown fields are silently dropped, and wrong types turn into nulls. So `drift.py` adds a second check on every line.

Detecting drift

`make_contract_check(contract)` returns a Python UDF that parses the raw line with `json.loads` and walks it against the contract:

- **Unknown key** -> an *unexpected* path, such as `ai_assisted` or the nested `owner.badge_counts`.
- **Value of the wrong type** -> a *type error*, such as `score` expected `int`, got `str: "five"`.
- **Not a JSON object** -> *malformed*.

Handling drift

Drift	Action	Why
New top-level column	**Evolve**: add it as a `STRING` column to the Bronze table with `mergeSchema`	Keeps new data queryable without a code change
New nested field	**Rescue**: store it in `_rescued_data` as JSON	Struct changes are risky to auto-evolve; the data is still kept
Type change	**Quarantine** the record	Loading `"five"` into an integer column would corrupt the data
Malformed line	**Quarantine**	Cannot be parsed
Missing primary key	**Quarantine**	Cannot be merged or de-duplicated

Quarantined records go to `so_bronze.quarantine_records` with the reason, the failing fields and the **untouched raw line**, so they can be replayed after a fix. Every drift observation is written to `so_ops.schema_drift_events` with the column, record count, an example value and the action taken. The batch itself finishes with status SUCCESS, and its log row shows how many records were quarantined.

Setting `drift_mode = rescue` keeps new top-level columns in `_rescued_data` instead of evolving the table. That is the more conservative choice when a data owner must approve every schema change.

Evolution in code

```
writer = good.write.format("delta").mode("overwrite").option("replaceWhere", ...)
if new_cols:
    writer = writer.option("mergeSchema", "true")   # Delta adds the new columns to the table
writer.saveAsTable(target)
```

Older batches simply have `NULL` in the new column. Later batches are also checked for previously evolved columns, so the column keeps filling.

Notebook `04_schema_drift_demo` builds a deliberately broken batch with five lines and loads it. Section 13 shows the result.

---

8. The Silver layer

Silver turns raw history into one trustworthy row per entity.

Step by step for questions

1. **Read** the selected batches from Bronze.
2. **De-duplicate** to one row per `question_id`, keeping the newest `_extracted_at`. `MERGE` requires unique source keys.
3. **Cast and clean**, in `transform_questions`:
   - Epoch seconds to timestamps: `F.timestamp_seconds(F.col("creation_date").cast("bigint"))` gives `created_at`.
   - HTML entities in titles are decoded: `Don't` becomes `Don't`.
   - `closed_reason` is normalised to a code such as `needs_details_or_clarity`.
   - Derived flags: `is_closed`, `has_any_answer`, `has_accepted_answer`, `is_migrated`.
   - Body features: length, `<pre>` blocks, `<code>` tags, links and images.
   - `tracked_tags`: the question's tags that are on our list of 18.
4. **Remove PII:**
   - `display_name`, `profile_image`, `link` and `account_id` are not selected, so they never reach Silver.
   - `user_id` becomes `owner_user_key = sha256(salt + ":" + user_id)`. The salt is a secret, so the key cannot be reversed by hashing all known ids. It is stable, so the same person always gets the same key, which keeps user analytics possible.
   - Bodies are not stored in the main tables. `silver.post_text_masked` keeps a copy with emails, IP addresses, password-like values and connection strings replaced by tokens such as `[EMAIL]`.
5. **Conform:** every column is cast to the exact type in the Silver contract, in contract order.
6. **Data quality:** rules such as "created_at is not null" and "activity is not before creation". Failing rows go to `silver.dq_quarantine` with the rule name.
7. **MERGE INTO** `silver.questions`, explained in the next section.

The other Silver tables follow the same pattern: `answers`, `users` built from post owners, `question_tags` as a bridge table, `question_snapshots` for metric history, and `post_text_masked`.

---

9. Idempotency and `MERGE INTO`, in depth

What MERGE does

```
MERGE INTO target t
USING source s
ON t.key = s.key
WHEN MATCHED AND <condition> THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...
```

For every source row, Delta finds the matching target row by key. Matched rows can be updated or deleted, unmatched rows are inserted, and all of it commits as one atomic transaction.

Our rule: newer AND different

Every Silver row stores the version it came from (`source_extracted_at`) and a hash of its business columns (`record_hash`):

```
MERGE INTO workspace.so_silver.questions AS t
USING src_questions AS s
ON t.question_id = s.question_id
WHEN MATCHED AND s.source_extracted_at > t.source_extracted_at
             AND s.record_hash <> t.record_hash THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT (...) VALUES (...)
```

Here is what happens in each situation:

Situation	Result
A brand-new question	Inserted
The same batch processed again	Same version, so the condition is false: **0 updates**
A newer batch with an identical question	Hash equal: no update; `load_timestamp` stays the same
A newer batch where the question changed	Updated, and `load_timestamp` records when
An **old** batch replayed after newer ones	Older version: no update. Old data can never overwrite new data

Together with de-duplicated sources, this guarantees **no duplicate keys** and **no change on re-runs**. The tests prove it with a checksum over every column of every Silver table, including `load_timestamp`, before and after re-processing all batches. The checksums are identical.

Four more MERGE patterns in the project

1. **Bridge table with deletes (`question_tags`).** Tags can be edited, so for questions whose current version came from this run, the source contains the new pairs marked `upsert` plus pairs that disappeared, marked `delete`:

```
WHEN MATCHED AND s._action = 'delete' THEN DELETE
WHEN NOT MATCHED AND s._action = 'upsert' THEN INSERT ...
```

2. **Slowly changing attributes (`users`).** Reputation is updated only from a newer extraction. `first_seen_at` uses `LEAST(...)` and `last_seen_at` uses `GREATEST(...)`, so any order of batches gives the same answer.
3. **Insert-only history (`question_snapshots`).** `WHEN NOT MATCHED THEN INSERT` only; a snapshot never changes.
4. **Soft deletes.** Deletion tombstones set `is_deleted = true` on questions and their answers. Rows are never physically removed, so the history stays auditable. A question that reappears in a newer extraction is automatically un-deleted.

Metrics have one extra twist. Votes and views do not change `last_activity_date`, so the incremental feed never sees them change. Weekly reconciliation snapshots carry fresh scores and views, and a MERGE applies a snapshot only when it is newer than `metrics_as_of`.

Where the row counts in the logs come from

After each MERGE we read the Delta transaction log:

```
spark.sql(f"DESCRIBE HISTORY {table} LIMIT 1")
# operationMetrics: numTargetRowsInserted, numTargetRowsUpdated, numTargetRowsDeleted
```

These numbers go straight into `rows_inserted`, `rows_updated` and `rows_deleted` in the execution log.

10. Parameterized backfills

Two modes

	`mode = incremental` **(daily)**	`mode = backfill` **(on demand)**
Raw-to-Bronze selects	Landing batches not loaded yet	Batches named by `batch_ids`, dated `start_date`..`end_date`, or at `landing_path`
Bronze-to-Silver selects	Bronze batches not yet in Silver	Batches named by `batch_ids` or dated `start_date`..`end_date`
Already-processed batches	Logged as SKIPPED	Processed again, which is safe because it is idempotent
Scope required?	No	Yes: a backfill with no scope is rejected, to prevent accidental reprocessing of everything

Examples

I want to...	Set
Run the normal daily load	`mode = incremental`
Redo one batch after fixing a bug	`mode = backfill`, `batch_ids = incr_20260925T155859Z`
Redo the last week	`mode = backfill`, `start_date = 2026-10-01`, `end_date = 2026-10-07`
Only full loads in that range	add `load_type = full`
Load a folder outside the normal layout	`landing_path = /Volumes/.../incr_...`

I want to...	Set
Load a period never extracted	Run `ingestion/full_load.py --from ... --to ...`, then an incremental run

The same parameters work as notebook widgets, as Databricks job parameters, and as command-line flags: `python -m so_lakehouse.cli bronze-to-silver --mode backfill --start-date 2026-10-01 --end-date 2026-10-07`.

11. Audit logging

The log table

`so_ops.pipeline_execution_logs` gets **one row per file in Raw-to-Bronze and one row per target table in Bronze-to-Silver**, for both full and incremental loads:

Column	Example
`run_id`	`raw_to_bronze-20261007T161502Z-3f9a1c`, shared by all rows of one run
`layer`	`Raw-to-Bronze` or `Bronze-to-Silver`
`run_mode`, `load_type`	`incremental`, `full`
`entity`, `batch_id`	`questions`, `full_2025-01-01_2025-04-01`
`source_parameter`	the file path, or the batch list and date range
`target_table`	`workspace.so_bronze.questions`
`start_time`, `end_time`, `duration_seconds`	timings in UTC
`status`	`SUCCESS`, `FAILED` or `SKIPPED`
`rows_read`, `rows_inserted`, `rows_updated`, `rows_deleted`, `rows_quarantined`	row metrics
`message`	error text, or notes such as "schema evolved: added ai_assisted"
`load_timestamp`	when the log row was written

How it is written

```
with audit.step(layer="Raw-to-Bronze", entity="questions", source_parameter=path,
                target_table=t, batch_id=b) as rec:
    rec.rows_inserted = ...        # the work happens here
# on exit: status SUCCESS, or FAILED with the error message; the row is always written
```

`AuditLogger.step` is a context manager. The log row is written in a `finally` block, so failures are logged with their error message and timings. The exception is then contained, and the run moves on to the next file. At the end of the run, if anything failed, the notebook raises an error, so the Databricks job shows as failed and can alert someone. The failure is already logged by then.

Notebook `03_audit_and_validation` queries the logs, the registry, drift events, quarantine, duplicate-key checks and the Delta MERGE history.

12. The data model in one page

The full dictionary, with every column, type, key and description, is in the README and in `docs/data_dictionary.md`. It is generated from the code.

Table	Primary key	Rows represent
`so_bronze.questions`	`question_id, _batch_id`	A question as received in one batch
`so_bronze.answers`	`answer_id, _batch_id`	An answer as received in one batch
`so_bronze.deleted_questions`	`question_id, _batch_id`	A deletion tombstone
`so_bronze.question_snapshots`	`question_id, snapshot_at, _batch_id`	A metric snapshot
`so_bronze.quarantine_records`	`quarantine_id`	A raw record that broke the contract
`so_silver.questions`	`question_id`	The current version of a question
`so_silver.answers`	`answer_id`	The current version of an answer

Table	Primary key	Rows represent
so_silver.question_tags	question_id, tag	A question-tag pair
so_silver.users	user_key	A pseudonymous user
so_silver.question_snapshots	question_id, snapshot_at	Metric history
so_silver.post_text_masked	post_type, post_id	Masked title and body; restricted
so_silver.dq_quarantine	dq_id	A record that failed a Silver rule
so_ops.pipeline_execution_logs	log_id	One processed file or table
so_ops.batch_registry	batch_id	A landing batch and its processing state
so_ops.schema_drift_events	event_id	One drift observation

Primary-key columns are `NOT NULL` in the DDL. On Unity Catalog, `00_setup` also declares informational `PRIMARY KEY` constraints.

13. Results from a real run

`scripts/demo_local_run.py` replays the whole scenario on local Spark and writes every audit row to `phase2_demo_results.md`. The headline numbers:

What we checked	Result
Initial Raw-to-Bronze, 5 batches, 16 files	16 SUCCESS log rows. Bronze holds 7,677 question rows, 9,701 answer rows, 7,385 snapshots and 1 deletion tombstone. 0 records quarantined
Initial Bronze-to-Silver	7,618 unique questions: 59 duplicates from the overlapping batches were merged away. Also 9,611 answers, 24,177 question-tag pairs, 8,332 pseudonymous users and 17,229 masked posts
Snapshots and deletions	Fresh scores and views applied to 7,282 questions. 1 deleted question and its 2 answers soft-deleted
Re-running the incremental pipeline	Every step logged SKIPPED; nothing written
Force-reprocessing all 5 batches	Bronze row counts unchanged. Every Silver MERGE inserted 0 and updated 0 rows. The checksum of all 6 Silver tables, over every column including `load_timestamp`, is identical
Date-range backfill for 2026-09-25	Exactly the 2 batches extracted that day were selected. Bronze replaced 6,316 rows of those batches; Silver changed 0 rows, because they are older than what Silver holds
Schema-drift batch	5 records read, 2 loaded, 3 quarantined (type mismatch, malformed JSON, missing key). Column `ai_assisted` added by mergeSchema; `owner.badge_counts` rescued. Status SUCCESS
Broken batch without manifest	Logged as FAILED with "missing _manifest.json"; all other batches unaffected
Audit log	94 rows written during the demo, one per file or table processed

The same scenario is automated in `tests/test_pipeline.py`. All 14 tests pass on Spark 3.5.3 with Delta Lake 3.2.1.

The same run on Databricks

On 10 October 2026 the pipeline ran on our Databricks Free Edition workspace as one job of nine notebook tasks on serverless compute. All nine succeeded, and the numbers match the local run:

What we checked on Databricks	Result
Initial Raw-to-Bronze	16 files, 24,764 records, 0 quarantined
Initial Bronze-to-Silver	7,618 questions, 9,611 answers, 24,177 tag pairs, 8,332 users
Re-run in incremental mode	Every step SKIPPED
Schema-drift batch	2 loaded, 3 quarantined, `ai_assisted` added through mergeSchema
Full Silver backfill	90,750 rows read, **0 inserted, 0 updated**
Duplicate primary keys	0 in every Silver table

The notebooks of that run, saved with their outputs, are in the repo's `evidence/` folder; `evidence/README.md` maps each rubric item to the notebook and section that proves it.

14. Running it yourself

On Databricks

1. Create a Git folder from the GitHub repository.
2. Create the secret `so_lakehouse/pii_salt` (README, Databricks setup).
3. Run `00_setup`, then `01_raw_to_bronze`, then `02_bronze_to_silver`.
4. Open `03_audit_and_validation` and walk through the evidence.
5. Run `04_schema_drift_demo`, then `03` again to see the drift events.
6. Demonstrate idempotency: run `01` and `02` again with `mode = backfill` and `start_date = 2026-09-25`, `end_date = 2026-10-07`. Then show in `03` that the duplicate counts are 0 and the new MERGE rows in the log show 0 inserted and 0 updated.

Locally

```
pip install -r requirements-dev.txt     # needs Java 17
PYTHONPATH=src pytest -q tests
PYTHONPATH=src python scripts/demo_local_run.py
```

---

15. Questions you may be asked, with answers

Why not use `inferSchema`? Inference guesses from a sample, can disagree between batches, and costs an extra scan. A contract makes types deliberate, versioned in Git and testable.

What happens if a column changes from integer to string? The contract check flags a type error, and the record goes to `quarantine_records` with the exact field and value. The rest of the batch loads, a drift event is recorded, and the log row shows the quarantined count.

And if the API adds a new field? With `drift_mode = evolve`, a new top-level field is added to the Bronze table as a column through `mergeSchema`. A new nested field is kept in `_rescued_data`. Nothing is lost and nothing crashes.

How do you know the pipeline is idempotent? Silver merges only newer-and-different rows, and the source is de-duplicated per key. The test suite reprocesses every batch and compares a checksum of every Silver table: they are identical, and every MERGE reports 0 inserted and 0 updated.

Is Bronze idempotent too? Yes. Each batch is written with `replaceWhere` on its own `_batch_id`, so a reload replaces it rather than appending a duplicate.

What is the difference between `load_timestamp` and `_extracted_at`? `_extracted_at` is when the API was called. `load_timestamp` is when our pipeline wrote that row. In Silver it changes only when the row is inserted or really updated.

How does a backfill avoid overwriting newer data with older data? Silver updates require `s.source_extracted_at > t.source_extracted_at`, so replaying an old batch cannot win.

How is PII handled? Names, avatars and profile links are dropped before Silver. User ids become salted SHA-256 keys, with the salt in a secret scope. Bodies are kept only in a restricted, masked table. Bronze keeps the raw data for replay, restricted to the team.

What if a whole file is broken? Its step is logged as FAILED with the error, other files and batches continue, and the run then raises so the job is marked failed. The registry keeps the batch as not loaded, so the next incremental run retries it.

Why snapshots? Votes and views do not change a question's last activity date, so the activity-based incremental feed would never see a new score. Snapshots from the weekly reconciliation bring those metrics in.

---

16. Exercises to deepen your understanding

1. In notebook 03, run `DESCRIBE HISTORY` on `so_silver.questions` and match each version to a log row by its timestamps and row counts.
2. Run `04_schema_drift_demo` with `drift_mode = rescue`, and compare `bronze.questions` with the evolve result.
3. Edit a question's title in a copy of a batch, land it as a new batch with a later `extracted_at_utc`, and run an incremental load. Check that `rows_updated = 1` and that `load_timestamp` changed only for that question.

4. Replay that edited batch with `mode = backfill`. Explain why `rows_updated` is now 0.
5. Use Delta time travel, `SELECT * FROM ... VERSION AS OF n`, to see a question before and after the update.
6. Add a data-quality rule, such as "score is not null", to `question_dq_rules()` in `silver.py`, and watch `silver.dq_quarantine`.

17. Glossary

Term	Meaning
Medallion architecture	Bronze for raw data, Silver for cleansed data, Gold for business-ready data
Delta Lake	Parquet files plus a transaction log, giving ACID transactions, MERGE, time travel and schema evolution
Schema-on-read	Applying a schema when the file is read, not when it is written
PERMISSIVE mode	Parser mode that keeps bad records and puts them in `_corrupt_record` instead of failing
Idempotent	Running it again with the same input changes nothing
Upsert / MERGE	Update if the key exists, insert if not, in one atomic step
`replaceWhere`	Overwrite only the rows matching a predicate
`mergeSchema`	Let a Delta write add new columns to the table
Quarantine	A side table for records that break the rules, kept for inspection and replay
Backfill	Re-processing historical data for a chosen period
Watermark	The point up to which data has been processed; ours is the batch registry
Soft delete	Marking a row deleted with a flag instead of removing it
Pseudonymisation	Replacing an identifier with a consistent token that cannot be reversed without a secret